

ЛАБОРАТОРНА РОБОТА №10

unit-тестування в мові Python

Мета роботи: ознайомитися з фреймворками для unit-тестування в мові Python

Хід роботи:

Завдання 1. Використовуючи фреймворки для unit-тестування unittest або pytest напишіть тести для класів створених під час виконання завдань 8 і 9 в лабораторній роботі №8. За потреби модифікуйте код класів для виправлення можливих помилок, що будуть знайдені під час покриття класів тестами. Онлайн-магазин (Завдання 8 з Л.Р. №8).

Лістинг коду:

```
import unittest

class Shop:
    def __init__(self, shop_name, store_type):
        self.shop_name = shop_name
        self.store_type = store_type
        self.number_of_units = 0

    def set_number_of_units(self, number):
        self.number_of_units = number

    def increment_number_of_units(self, count):
        self.number_of_units += count

class Discount(Shop):
    def __init__(self, shop_name, store_type):
        super().__init__(shop_name, store_type)
        self.discount_products = []

class TestShop(unittest.TestCase):
    def setUp(self):
        self.store = Shop("Тест-Маг", "Продукти")
```

					ДУ «Житомирська політехніка».25.F2.02.000 – Лр10			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Гришкова Е. А.			Звіт з лабораторної роботи		Лім.	Арк.
Перевір.		Желізко В. В.						1
Керівник								4
Н. контр.							ФІКТ Гр. ВТк-25-1	
Зав. каф.								

```

def test_initialization(self):
    self.assertEqual(self.store.shop_name, "Тест-Маг")
    self.assertEqual(self.store.store_type, "Продукти")
    self.assertEqual(self.store.number_of_units, 0)

def test_set_units(self):
    self.store.set_number_of_units(100)
    self.assertEqual(self.store.number_of_units, 100)

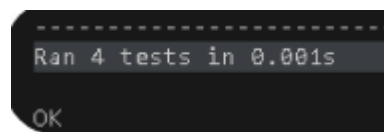
def test_increment_units(self):
    self.store.set_number_of_units(50)
    self.store.increment_number_of_units(10)
    self.assertEqual(self.store.number_of_units, 60)

def test_discount_inheritance(self):
    ds = Discount("Акція", "Одяг")
    self.assertEqual(ds.discount_products, [])
    self.assertEqual(ds.shop_name, "Акція")

if __name__ == "__main__":
    unittest.main()

```

Результат виконання:



У межах тестування класу Shop було реалізовано методи для перевірки коректності ініціалізації об'єкта та роботи з асортиментом. Використання setUp дозволяє створювати чистий екземпляр класу перед кожним тестом. Тести перевіряють як пряму зміну атрибута number_of_units, так і роботу методів set_number_of_units та increment_number_of_units. Окремий тест підтверджує, що клас Discount правильно успадковує атрибути базового класу

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр10	Арк.
		Желіско В.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		2

Завдання 2. Облік користувачів на сайті (Завдання 9 з Л.Р. №8).

Лістинг коду:

```
import unittest

class User:
    def __init__(self, first_name, last_name, nickname):
        self.first_name = first_name
        self.last_name = last_name
        self.nickname = nickname
        self.login_attempts = 0

    def increment_login_attempts(self):
        self.login_attempts += 1

    def reset_login_attempts(self):
        self.login_attempts = 0

class Privileges:
    def __init__(self):
        self.privileges = ["Allowed to add message"]

class Admin(User):
    def __init__(self, first_name, last_name, nickname):
        super().__init__(first_name, last_name, nickname)
        self.priv = Privileges()

class TestUser(unittest.TestCase):
    def setUp(self):
        self.user = User("Еліна", "Тестовий", "elina_test")

    def test_full_name(self):
        self.assertEqual(self.user.first_name, "Еліна")
        self.assertEqual(self.user.last_name, "Тестовий")

    def test_login_attempts(self):
        self.user.increment_login_attempts()
        self.user.increment_login_attempts()
        self.assertEqual(self.user.login_attempts, 2)
        self.user.reset_login_attempts()
        self.assertEqual(self.user.login_attempts, 0)

    def test_admin_privileges(self):
        admin = Admin("Ольга Ніщенко", "Адмін", "boss")
        self.assertIn("Allowed to add message", admin.priv.privileges)

if __name__ == "__main__":
    unittest.main()
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр10	Арк.
		Желізко В.В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання:

```
...
-----
Ran 3 tests in 0.001s

OK
```

Модульні тести перевіряють роботу лічильника `login_attempts`: послідовне збільшення значення та його повне обнулення через метод `reset_login_attempts`. Тест для класу `Admin` підтверджує успішну композицію об'єктів, перевіряючи наявність специфічних привілеїв у вкладеному об'єкті `Privileges`

Посилання на репозиторій: <https://github.com/wghtl4s/Python>

Висновки: Під час виконання лабораторної роботи було опановано методику тестування програмного коду за допомогою фреймворку `unittest`

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр10	Арк.
		Желізко В.В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		